```
import tensorflow as tf
print(tf.__version__)
```

```
2.20.0
```

```
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
import numpy as np
```

```
import tensorflow_datasets as tfds

(train_ds, val_ds), ds_info = tfds.load(
    'cats_vs_dogs',
    split=['train[:80%]', 'train[80%:]'],
    as_supervised=True,
    with_info=True
)

print(ds_info)
```

```
WARNING:absl:Variant folder /root/tensorflow_datasets/cats_vs_dogs/4.0.1 has no dataset_info.json
Downloading and preparing dataset Unknown size (download: Unknown size, generated: Unknown size, total: Unknown size) to /root/t
Dl Completed...: 100%  1/1 [00:11<00:00, 11.45s/ url]
Dl Size...: 100%  786/786 [00:11<00:00, 64.20 MiB/s]

WARNING:absl:1738 images were corrupted and were skipped

Dataset cats_vs_dogs downloaded and prepared to /root/tensorflow_datasets/cats_vs_dogs/4.0.1. Subsequent calls will reuse this d
tfds.core.DatasetInfo(
    name='cats_vs_dogs',
    full_name='cats_vs_dogs/4.0.1',
    description="""
    A large set of images of cats and dogs. There are 1738 corrupted images that are dropped.
    """,
    homepage='https://www.microsoft.com/en-us/download/details.aspx?id=54765',
    data_dir='/root/tensorflow_datasets/cats_vs_dogs/4.0.1',
    file_format=tfrecord,
    download_size=786.67 MiB,
    dataset_size=1.04 GiB,
    features=FeaturesDict({
        'image': Image(shape=(None, None, 3), dtype=uint8),
        'image/filename': Text(shape=(), dtype=string),
        'label': ClassLabel(shape=(), dtype=int64, num_classes=2),
    }),
    supervised_keys=('image', 'label'),
    disable_shuffling=False,
    nondeterministic_order=False,
    splits={
        'train': <SplitInfo num_examples=23262, num_shards=16>,
    },
    citation="""@Inproceedings (Conference){asirra-a-captcha-that-exploits-interest-aligned-manual-image-categorization,
    author = {Elson, Jeremy and Douceur, John (JD) and Howell, Jon and Saul, Jared},
    title = {Asirra: A CAPTCHA that Exploits Interest-Aligned Manual Image Categorization},
    booktitle = {Proceedings of 14th ACM Conference on Computer and Communications Security (CCS)},
    year = {2007},
    month = {October},
```

```
import tensorflow as tf

IMG_SIZE = 150

def preprocess(image, label):
    image = tf.image.resize(image, (IMG_SIZE, IMG_SIZE))
    image = image / 255.0
    return image, label

train_ds = train_ds.map(preprocess)
val_ds = val_ds.map(preprocess)
```

```
train_ds = train_ds.batch(32)
val_ds = val_ds.batch(32)
```

```
for images, labels in train_ds.take(1):
    print("Image Shape:", images.shape)
    print("Label Shape:", labels.shape)
```

```
Image Shape: (32, 150, 150, 3)
Label Shape: (32,)
```

```
from tensorflow.keras import layers, models

model = models.Sequential([

    layers.Conv2D(
        32,
        (3,3),
        activation='relu',
        input_shape=(150,150,3)
    ),
    layers.MaxPooling2D(),

    layers.Conv2D(
        64,
        (3,3),
        activation='relu'
    ),
    layers.MaxPooling2D(),

    layers.Conv2D(
        128,
        (3,3),
        activation='relu'
    ),
    layers.MaxPooling2D(),

    layers.Flatten(),

    layers.Dense(
        128,
        activation='relu'
    ),

    layers.Dropout(0.5),

    layers.Dense(
        1,
        activation='sigmoid'
    )
])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shap
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 148, 148, 32)	896
max_pooling2d (MaxPooling2D)	(None, 74, 74, 32)	0
conv2d_1 (Conv2D)	(None, 72, 72, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 36, 36, 64)	0
conv2d_2 (Conv2D)	(None, 34, 34, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 17, 17, 128)	0
flatten (Flatten)	(None, 36992)	0
dense (Dense)	(None, 128)	4,735,104
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 1)	129

Total params: 4,828,481 (18.42 MB)
Trainable params: 4,828,481 (18.42 MB)
Non-trainable params: 0 (0.00 B)

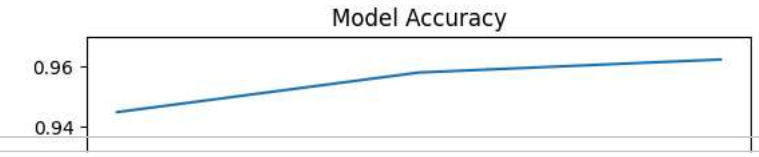
```
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=3
)
```

```
Epoch 1/3
582/582 ━━━━━━━━━━━━━━━━━━━━ 939s 2s/step - accuracy: 0.9449 - loss: 0.1406 - val_accuracy: 0.8235 - val_loss: 0.5848
Epoch 2/3
582/582 ━━━━━━━━━━━━━━━━━━━━ 933s 2s/step - accuracy: 0.9581 - loss: 0.1101 - val_accuracy: 0.8220 - val_loss: 0.6553
Epoch 3/3
582/582 ━━━━━━━━━━━━━━━━━━━━ 918s 2s/step - accuracy: 0.9624 - loss: 0.0967 - val_accuracy: 0.8123 - val_loss: 0.6985
```

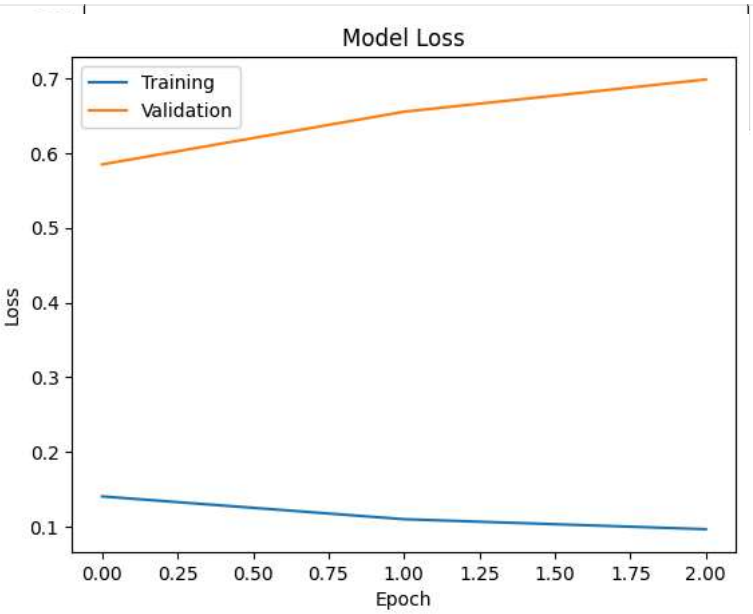
```
loss, accuracy = model.evaluate(val_ds)

print("Validation Loss:", loss)
print("Validation Accuracy:", accuracy)
```

```
146/146 ━━━━━━━━━━━━━━━━━━━━ 71s 480ms/step - accuracy: 0.8123 - loss: 0.6985
Validation Loss: 0.698460578918457
Validation Accuracy: 0.8123387694358826
```

```
import matplotlib.pyplot as plt

plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])

plt.title('Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Training', 'Validation'])

plt.show()
```




```
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])

plt.title('Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Training', 'Validation'])

plt.show()
```




```
model.save("cnn_image_classifier.h5")
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format
```

```
from google.colab import files
files.download("cnn_image_classifier.h5")
```